

1. Executive Summary

TicketSense is an AI-powered customer support triage system designed to automatically analyze incoming support tickets and classify them based on **sentiment, urgency, and churn risk**.

It helps support teams prioritize critical issues, generate **AI-assisted draft responses**, and trigger alerts for high-risk customers, ensuring faster and more efficient customer service operations.

The system is built with a **fault-tolerant AI architecture**, ensuring uninterrupted service even when primary LLM providers fail.

2. Objectives

- Automate ticket classification (sentiment, urgency, churn risk)
 - Reduce response time using AI-generated draft replies
 - Identify high-risk customers early (churn prevention)
 - Provide real-time analytics dashboard for support teams
 - Ensure system reliability using multi-model fallback architecture
-

3. System Architecture

TicketSense follows a **modular, service-oriented architecture**:

User Input → API Layer (FastAPI) → AI Orchestrator → LLM Providers → Response Engine → Storage + Dashboard

Core Components:

- **API Layer:** FastAPI handles incoming requests
 - **AI Orchestrator:** Routes requests to LLM providers
 - **Storage Layer:** SQLite database for ticket persistence
 - **Dashboard Layer:** Streamlit admin dashboard
 - **Demo Interface:** Gradio UI for quick interaction
 - **Notification Layer:** Discord alerts for high-risk cases
-

4. Tech Stack

Backend

- Python 3.10+

- FastAPI (REST API)
- Pydantic Settings (configuration management)
- Requests (HTTP handling)

Frontend / UI

- Gradio (demo interface)
- Streamlit (admin dashboard)

AI Models

- Groq (Llama 3 family)
- Google Gemini (fallback provider)

Database

- SQLite (lightweight persistent storage)

DevOps / Tools

- Python-dotenv (.env management)
 - Pytest (testing framework)
 - Uvicorn (ASGI server)
-

5. AI & Model Integration

TicketSense uses a **multi-provider AI system**:

Primary Flow:

1. Groq Llama model (GROQ_MODEL)
2. Fallback Groq model (GROQ_MODEL_FALLBACK)
3. Google Gemini API (secondary fallback)
4. Static safe response (final fallback)

Use Cases:

- Ticket classification (sentiment, urgency, churn risk)
 - Response drafting
 - Risk scoring and prioritization
-

6. AI Fallback Architecture

To ensure **high availability and fault tolerance**, the system implements a layered fallback strategy:

Workflow:

1. Request sent to primary Groq model
2. If failure occurs:
 - Retry with fallback Groq model
3. If still failing:
 - Switch to Google Gemini
4. If all models fail:
 - Return a safe default structured response

Failure Conditions Handled:

- Rate limits (HTTP 429)
 - Server errors (HTTP 500/503)
 - Timeout errors
 - Invalid JSON responses
-

7. Key Features

7.1 Ticket Classification

- Sentiment: Positive / Neutral / Negative
- Urgency: Low / Medium / High / Critical
- Churn Risk: Probability score
- Category tagging

7.2 AI Response Generation

- Draft customer replies
- Context-aware responses
- Tone adaptation

7.3 Real-Time Dashboard

- Ticket monitoring
- Risk visualization

- System performance tracking

7.4 Alert System

- Discord webhook notifications
- Triggered for high-risk tickets

7.5 Robust Error Handling

- JSON parsing recovery
 - Retry mechanisms
 - Structured logging
-

8. Project Structure

```
TicketSense/  
|  
├── main.py          # FastAPI endpoints  
├── classifier.py    # AI classification logic  
├── drafter.py       # Response generation  
├── models.py        # Pydantic schemas  
├── config.py        # Environment configuration  
├── database.py      # SQLite operations  
├── dashboard.py     # Streamlit dashboard  
├── app_gradio.py    # Gradio demo UI  
├── discord_alert.py # Notification system  
├── test_api.py      # Unit tests  
|  
├── ticketsense.db   # SQLite database  
├── requirements.txt  # Dependencies  
└── .env             # Secrets (not committed)
```

9. API Endpoints

POST /analyze

Analyzes a single ticket.

Request:

```
{  
  "text": "My internet is not working and I need urgent help"  
}
```

Response:

- sentiment

- urgency
 - churn risk
 - category
 - explanation
-

POST /analyze/batch

Processes multiple tickets in bulk.

POST /draft-response

Generates AI-powered customer reply.

10. Error Handling & Resilience

The system is designed for production-grade reliability:

- API timeout handling (8–10 seconds)
 - Automatic retry mechanisms
 - Regex-based JSON recovery for malformed outputs
 - Graceful degradation to fallback responses
 - Logging for every AI provider switch
-

11. Security Considerations

- API keys stored in .env
 - Managed via pydantic-settings
 - No hardcoded secrets
 - Network requests use bounded timeouts
 - .env excluded from version control
-

12. Performance Considerations

- Lightweight SQLite for fast local operations
- Async-ready FastAPI architecture
- Efficient prompt engineering to reduce token usage
- Model fallback prevents downtime

13. Testing Strategy

- Unit tests using pytest
- API validation using TestClient
- Focus areas:
 - Endpoint correctness
 - JSON response structure
 - Failure handling paths

14. Future Enhancements

- PostgreSQL migration for scale
- Redis caching layer
- Prometheus + Grafana monitoring
- Role-based admin dashboard
- Fine-tuned custom LLM for support domain
- Email + Slack integrations
- Advanced analytics (ticket clustering)

15. Conclusion

TicketSense demonstrates a **production-ready AI system design** combining LLM orchestration, fault tolerance, and real-time monitoring.

It is suitable for:

- Customer support automation
- SaaS helpdesk optimization
- AI engineering portfolio / placement showcase